Benchmark-Aufgabe 04
4-Bit-Akkumulator-CPU

Entwurfsaufgabe, Prüfgrundlagen und Ergebnisse von drei realen Läufen des VHDL-Multiagentensystems bis zum Flashen auf das Arty S7-50

Stand	12.10.2026
Aufgabe	`Schaltungsagent/benchmark/prompts/04_mini_cpu.md`
Referenz-Testbench	`Schaltungsagent/benchmark/reference/04_mini_cpu_ref_tb.vhd`
Ergebnisse	`docs/benchmark/ergebnisse/20261012-04_mini_cpu/`
Modell	opencode/muse-spark-1.3-contributor-free (Fallback: mimo-v2.6-flash-free)
Zielhardware	Digilent Arty S7-50 (xc7s50csga324-1), Vivado 2026.1

> **Ergebnis in Kürze.** Alle drei Läufe erzeugten ohne menschlichen Eingriff einen funktionsfähigen Entwurf mit zehn Modulen, bestanden alle Prüfungen des Systems und die unabhängige Referenz-Testbench, erfüllten das Timing und wurden auf das Board geflasht; die LED-Ausgabe hat der Benutzer am Board bestätigt. Die ersten beiden Läufe (37:42 und 38:00 min) kamen ohne Reparatur aus. Aus ihnen wurden zehn Verbesserungen abgeleitet (#48–#57); der Kontrolllauf (47:06 min) bestätigt sie und zeigt, dass der differentielle Test eine Abweichung von einem Takt findet, die die Referenz-Testbench mit ihrem Spielraum nicht sieht.

Inhalt

1. Ziel und Einordnung
2. Aufgabenstellung
3. Erwartetes Verhalten
4. Prüfgrundlagen
5. Versuchsaufbau
6. Ergebnisse der drei Läufe
7. Verlauf der einzelnen Läufe
8. Befunde und Verbesserungen
9. Vergleich mit einem einzelnen OpenCode-Aufruf
10. Grenzen

11. Ablage und Reproduktion

1 Ziel und Einordnung

Die Benchmark-Aufgaben 01–03 (Zähler, Hamming-Decoder, Ampel) bestehen aus einem bis drei Modulen. Aufgabe 04 prüft, ob das Multiagentensystem (MAS) eine deutlich größere, hierarchische Schaltung von der Anforderung bis zur Hardware umsetzen kann. Gewählt wurde eine kleine Akkumulator-CPU als Vorstufe einer 8-Bit-CPU: Sie enthält die drei Grundbausteine eines Rechners – **Rechenwerk** (ALU mit Flags), **Steuerwerk** (Befehlsablauf mit Befehlszähler und Befehlsregister) und **Speicher** (Programm- und Datenspeicher) – und arbeitet mit 4 Bit breiten Daten, damit jedes Ergebnis direkt an den vier LEDs des Boards ablesbar ist.

Zwei Anforderungen bestimmten die Formulierung: Erstens muss das Ergebnis am Board **von einem Menschen nachvollziehbar** sein (festes Demoprogramm mit bekannter LED-Folge, wählbare Anzeige, Einzelschritt). Zweitens muss die Aufgabe **taktgenau eindeutig** sein, weil das MAS einen differentiellen Test gegen ein ausführbares Referenzmodell durchführt, das mit zufälligen Eingangsfolgen Takt für Takt vergleicht. Jede Latenz (Synchronisierer, Entprellung, Tick, Befehlsphasen, LED-Register) ist deshalb ausdrücklich festgelegt.

2 Aufgabenstellung

Der vollständige Wortlaut steht in `04_mini_cpu.md`; hier die Bestandteile in Tabellenform.

2.1 Schnittstelle und Board-Belegung

Port	Typ	Board	Bedeutung
clk	in std_logic	R2, 100 MHz	Systemtakt
rst	in std_logic	BTN0 (G15)	synchroner Reset, active high
btn_step	in std_logic	BTN1 (K16)	Einzelschritt
sw(1 downto 0)	in	SW0, SW1	Anzeige: 00 OUT, 01 ACC, 10 PC, 11 Status
sw(2)	in	SW2	0 Automatik (ein Befehl je Tick), 1 Einzelschritt
led(3 downto 0)	out	LD5 … LD2	Anzeigewert, led(0) = LD2
TICK_CYCLES	Generic	–	Standard 50 000 000 (2 Befehle/s), mindestens 4
DEBOUNCE_CYCLES	Generic	–	Standard 1 000 000 (10 ms)

2.2 Befehlssatz

Befehlswort 8 Bit: Opcode in Bit 7..4, Operand (Adresse a, Konstante n oder Sprungziel t) in Bit 3..0. Akkumulator ACC, Befehlszähler PC und Ausgaberegister OUT haben 4 Bit, das Befehlsregister IR 8 Bit; Z und C sind Flag-Register. „Z“ bedeutet: Z := 1 genau dann, wenn der neue ACC-Wert 0 ist. Nicht genannte Flags bleiben unverändert – auch C bei LDI und LDA.

Opcode	Befehl	Wirkung	Flags
0	NOP	keine Wirkung	–
1	LDI n	ACC := n	Z
2	LDA a	ACC := RAM[a]	Z
3	STA a	RAM[a] := ACC	–
4	ADD a	ACC := (ACC + RAM[a]) mod 16	Z, C (Übertrag)
5	SUB a	ACC := (ACC − RAM[a]) mod 16	Z, C (Borgen)
6–8	AND/OR/XOR a	bitweise mit RAM[a]	Z, C := 0
9	SHL	ACC links, '0' nachschieben	Z, C := altes Bit 3
A	SHR	ACC rechts, '0' nachschieben	Z, C := altes Bit 0
B	JMP t	PC := t	–
C	JZ t	PC := t, wenn Z = 1	–
D	JC t	PC := t, wenn C = 1	–
E	OUT	OUT := ACC	–
F	HLT	anhalten bis Reset	–

2.3 Demoprogramm im Programmspeicher

RAM[0] und RAM[1] halten die letzten beiden Fibonacci-Zahlen a und b. Jeder Schleifendurchlauf berechnet a + b, gibt die Summe aus und verschiebt das Paar. Läuft die Summe über (21 > 15), setzt ADD das Carry, JC springt zum HLT.

Adresse	Wort	Befehl	Bedeutung
0	x"11"	LDI 1	ACC = 1
1	x"30"	STA 0	a := 1
2	x"31"	STA 1	b := 1
3	x"E0"	OUT	OUT = 1
4	x"20"	LDA 0	Schleife: ACC := a
5	x"41"	ADD 1	ACC := a + b
6	x"DE"	JC 14	Überlauf → Ende
7	x"32"	STA 2	s := a + b
8	x"21"	LDA 1	
9	x"30"	STA 0	a := b
10	x"22"	LDA 2	
11	x"31"	STA 1	b := s
12	x"E0"	OUT	neue Zahl ausgeben
13	x"B4"	JMP 4	
14	x"F0"	HLT	Ende
15	x"00"	NOP	

2.4 Steuerwerk

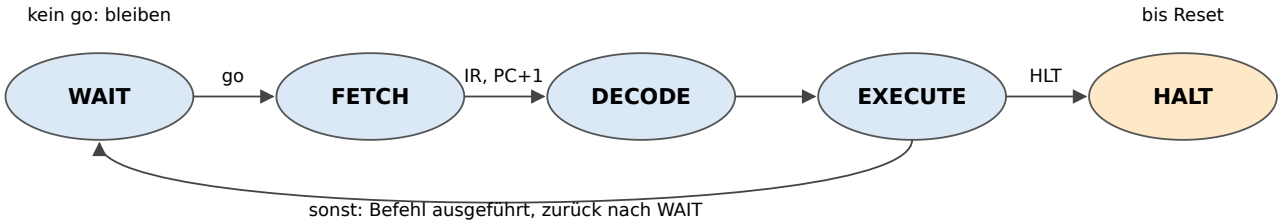


Abbildung 1: Zustandsautomat des Steuerwerks. go ist in Automatik der Tick, im Einzelschritt der entprellte Tastendruck; in anderen Zuständen als WAIT wird go ignoriert.

Ein Befehl braucht nach go genau drei weitere Taktflanken: FETCH lädt IR := ROM[PC] und erhöht PC, DECODE ändert kein Register, EXECUTE führt den Befehl aus (ein Sprung überschreibt das PC + 1 aus FETCH).

2.5 Eingänge, Takt und Anzeige

- **Synchronisierung:** sw und btn_step je über zwei Flipflops.
- **Entprellung:** Register dout und Zähler; dout übernimmt den Eingang mit der DEBOUNCE_CYCLES-ten aufeinanderfolgenden Flanke, an der Eingang und dout verschieden sind. Der Schrittimpuls rise ist ein Register und genau einen Takt lang 1, beginnend mit der Flanke, an der dout von 0 auf 1 wechselt.
- **Tick:** Zähler 0 … TICK_CYCLES−1, Tick beim Zählerstand TICK_CYCLES−1 für genau einen Takt; der erste Befehl beginnt nach dem Reset mit der TICK_CYCLES-ten steigenden Flanke.
- **Anzeige:** led ist ein Register; eine Schalteränderung ist nach genau drei Flanken sichtbar, eine Registeränderung der CPU eine Flanke später. Status: led(3) HALT, led(2) Betriebsart, led(1) C, led(0) Z.
- **Reset:** alle Register einschließlich der 16 RAM-Zellen werden 0, das Steuerwerk geht nach WAIT.

2.6 Modulstruktur

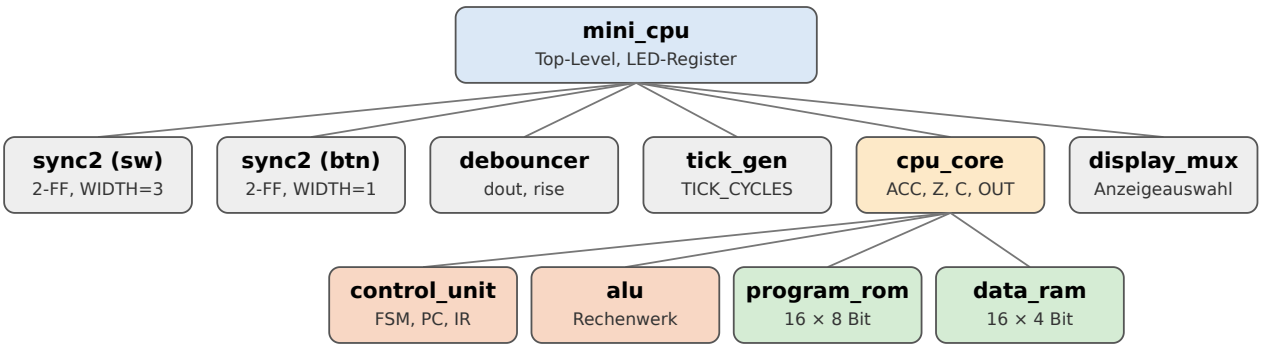


Abbildung 2: Vorgegebene Hierarchie (zwei Ebenen, zehn Entities, je eine Datei `src/<entity>.vhd`). Orange: Rechen- und Steuerwerk, grün: Speicher.

Gefordert ist außerdem eine Testbench `tests/mini_cpu_tb.vhd` mit TICK_CYCLES = 4 und DEBOUNCE_CYCLES = 2, die die OUT-Folge, das Anhalten, den Einzelschritt mit Entprellung, einen Reset mitten im Programm und die Latenz von drei Flanken nach einer Schalteränderung prüft.

3 Erwartetes Verhalten

Das Demoprogramm wurde vorab mit einem Befehlssatzmodell in Python ausgeführt (`pruefgrundlagen/befehlssatz_modell.py`). Die CPU führt 58 Befehle aus und hält dann an; bei 2 Befehlen je Sekunde ergeben sich folgende Ausgaben:

nach Befehl Nr.	OUT (dezimal)	LEDs LD5…LD2	Zeit nach Reset [s]
4	1	0001	2,0
13	2	0010	6,5
23	3	0011	11,5
33	5	0101	16,5
43	8	1000	21,5
53	13	1101	26,5

Endzustand nach dem HLT (58. Befehl, nach rund 29 s): OUT = 13 (1101), ACC = 5 (0101; 8 + 13 = 21 ≡ 5), PC = 15 (1111, nach dem Holen des HLT an Adresse 14), Z = 0, C = 1, Status in Automatik 1010.

3.1 Prüfanleitung fürs Board

Einstellung	Anzeige	Beobachtung	nach dem Halt
alle Schalter unten, BTN0	OUT	0001, 0010, 0011, 0101, 1000, 1101 (etwa alle 5 s), Halt nach rund 29 s	`1101`
SW0 oben	ACC	ändert sich zweimal je Sekunde	`0101`
SW1 oben	PC	Adresse des nächsten Befehls	`1111`
SW1 und SW0 oben	Status (HALT, Modus, C, Z)	LD5 geht beim Halt an	`1010`
SW2 oben, BTN1 drücken	gewählte Anzeige	genau ein Befehl je Druck	–

> Nach BTN0 beginnt das Programm von vorn. Im Einzelschritt (SW2 oben) zeigt die ACC-Anzeige nach dem ersten Druck 0001, die PC-Anzeige 0001; ein kurzes Prellen löst keinen Schritt aus.

4 Prüfgrundlagen

4.1 Unabhängige Referenz-Testbench

`04_mini_cpu_ref_tb.vhd` ist dem System unbekannt und prüft nur über die Ports der Top-Level-Entity: die Folge der OUT-Werte (Reihenfolge exakt, Zeitpunkte mit Spielraum), den Endzustand in allen vier Anzeigen, das dauerhafte Anhalten, einen Reset mitten im Programm sowie den Einzelschritt (kein Schritt ohne Druck, keiner durch einen 1-Takt-Impuls, je Druck genau ein

Befehl). Sie wurde gegen eine eigene, vom System unabhängige Umsetzung geprüft (`pruefgrundlagen/eigene_umsetzung_mini_cpu.vhd`) und gegen sechs absichtlich fehlerhafte Varianten:

eingebauter Fehler	**erkannt**
ADD setzt kein Carry (kein Halt)	ja – zusätzliche OUT-Werte
rise auch beim Loslassen der Taste	ja – PC nach dem ersten/zweiten Schritt
keine Entprellung	ja – Schritt durch 1-Takt-Impuls
HLT ohne Wirkung	ja – zusätzliche OUT-Werte
Z und C in der Statusanzeige vertauscht	ja – Status nach dem Anhalten
LDA löscht C	nein – mit dem festen Programm an den Ports nicht beobachtbar

Grenze: Weil das Programm fest im ROM steht, sind Fehler in Befehlen, die das Demoprogramm nicht in der kritischen Reihenfolge nutzt, von außen nicht sichtbar. Die Testbench toleriert außerdem bewusst Abweichungen im Zeitverhalten; taktgenaue Prüfung leistet der differentielle Test des Systems.

4.2 Referenzmodelle des Systems

Das MAS lässt vor der Implementierung ein Python-Referenzmodell aus PROMPT.md schreiben und vergleicht den Entwurf in einer erzeugten Testbench Takt für Takt damit (TICK_CYCLES = 4, DEBOUNCE_CYCLES = 2, zufällige Eingangsfolgen mit Reset-Fenstern). Jedes in den Läufen erzeugte Modell wurde während des Laufs, ohne Eingriff, gegen die eigene Umsetzung geprüft: alle stimmten mit 4 bis 7 Zufallsfolgen à 400 Takten überein, auch das im Kontrolllauf nach einem Modellstreit neu erzeugte.

5 Versuchsaufbau

- **Eingang:** Server wie mit `start.py`, Aufgabe als `POST /api/chat` mit dem Wortlaut von `04_mini_cpu.md` – derselbe Weg wie aus dem Dashboard. Ein zweiter WebSocket-Client schrieb alle Ereignisse mit.
- **Kein Eingriff:** Während der Läufe wurde weder am Systemcode noch im Projekt etwas geändert; Beobachtungen (Gegenprobe der Referenzmodelle, Lesen von Logs) liefen in Kopien.
- **Einstellungen:** `repair_policy: persistent`, zwei parallele HDL-Worker, Zeitlimit 900 s je OpenCode-Aufruf, Synthese mit Timing-Gate, Flash in SRAM und QSPI. Kein UART, also kein automatischer Hardwaretest.
- **Codestände:** Lauf 1 und 2 mit `af7d863` (vor den Verbesserungen), Kontrolllauf mit `256a317` (Verbesserungen #48–#56; #57 entstand aus diesem Lauf).

6 Ergebnisse der drei Läufe

	Lauf 1	Lauf 2	Kontrolllauf
Projekt	`fibonacci-accumulator-cpu-c71b1f`	`mini-accumulator-cpu-66e33d`	`fibonacci-accu-cpu-f8fb57`
Codestand	af7d863 (Stand vor #48-#57)	af7d863 (Stand vor #48-#57, Wiederholung)	256a317 (Stand mit #48-#56)
Gesamtdauer	37:42 min	38:00 min	47:06 min
Ergebnis	COMPLETED	COMPLETED	COMPLETED
Reparaturen	0	0	2
OpenCode-Aufrufe (Summe)	22 (42,5 min)	22 (41,8 min)	24 (44,0 min)
Einzelanforderungen	91; Abschlussprüfung: alle erfüllt	74; Abschlussprüfung: alle erfüllt	91; Abschlussprüfung: alle erfüllt
Testbenches	9	9	10 (mit cpu_core_tb)
Mutationsquote	0,9	0,7	1,0
Timing (WNS)	+3,75 ns	+3,14 ns	+3,77 ns
LUTs / Flipflops	95 / 151	108 / 149	93 / 151
Referenz-Testbench	bestanden	bestanden	bestanden
Board	SRAM + QSPI geflasht	SRAM + QSPI geflasht	SRAM + QSPI geflasht
Ereignisse (Lebenszeichen)	176 (102), 179 KB	177 (103), 165 KB	136 (20), 57 KB
task_graph.json	1035 KB	996 KB	27 KB

6.1 Dauer der Abschnitte

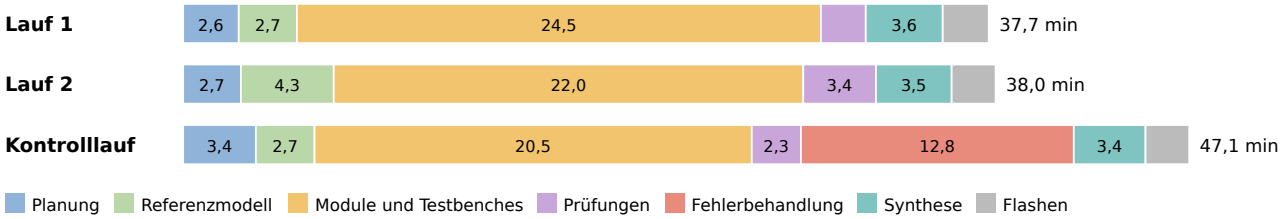


Abbildung 3: Dauer der Abschnitte in Minuten. „Fehlerbehandlung“ ist die Zeit zwischen Implementierung und Synthese, die nicht auf die Prüfungen selbst entfällt (Diagnose, Reparatur, Neuerzeugung des Modells).

Abschnitt [min]	Lauf 1	Lauf 2	Kontrolllauf
Planung	2,6	2,7	3,4
Referenzmodell	2,7	4,3	2,7
Module und Testbenches	24,5	22,0	20,5
Prüfungen	2,1	3,4	2,3
Fehlerbehandlung	0,0	0,0	12,8
Synthese	3,6	3,5	3,4
Flashen	2,2	2,1	2,0
Summe	**37,7**	**38,0**	**47,1**

Die Implementierung der zehn Module samt Testbenches dominiert jeden Lauf. Zwischen Lauf 1/2 und dem Kontrolllauf sank sie trotz einer zusätzlichen Testbench von 24,5 bzw. 22,0 auf 20,5 Minuten (Abschnitt 8). Prüfungen dauern dank GHDL und vorzeitig endender Abschlussprüfung nur 2–3 Minuten, Synthese und Flashen zusammen rund 5,5 Minuten.

6.2 Ablauf der Implementierung

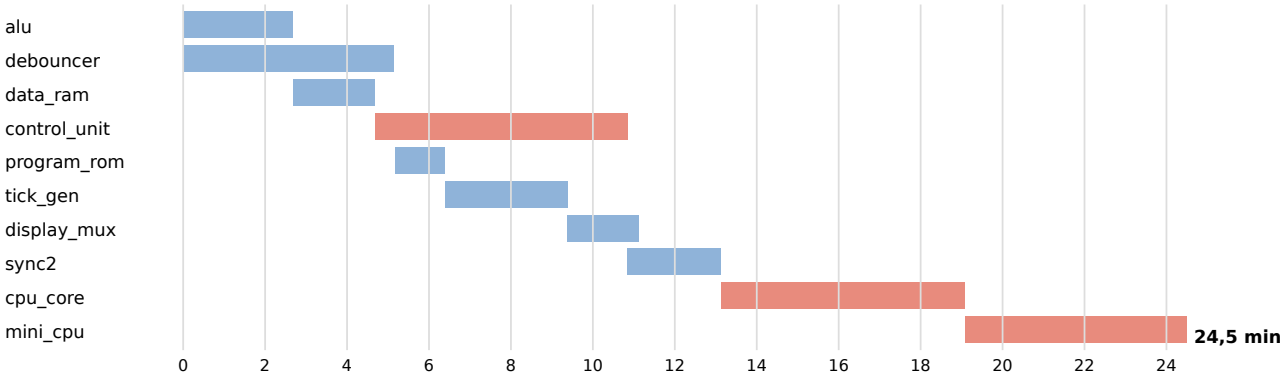


Abbildung 4a: Lauf 1 – paketweise Planung (Minuten ab Beginn der Implementierung, rot: kritischer Pfad control_unit → cpu_core → mini_cpu). control_unit startet als viertes Modul, cpu_core erst, nachdem das ganze erste Paket fertig ist.

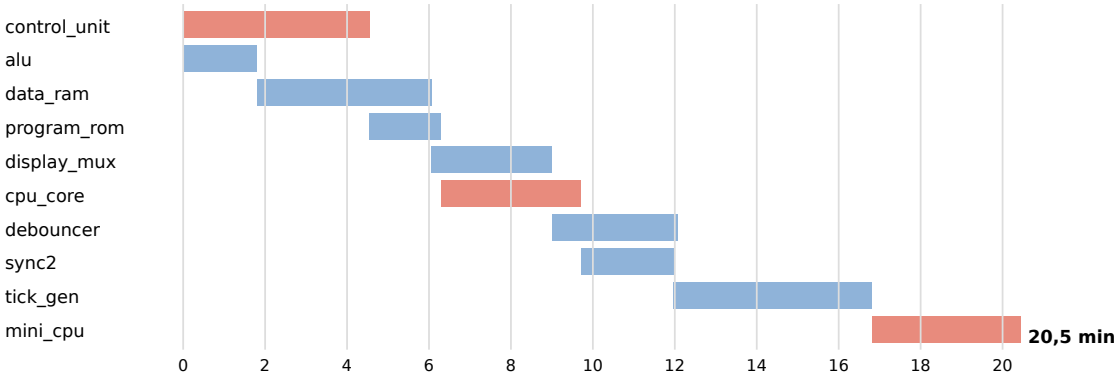


Abbildung 4b: Kontrolllauf – kritischer Pfad zuerst (#48). control_unit startet sofort, cpu_core sobald seine vier Abhängigkeiten fertig sind; die übrigen Blattmodule füllen die freien Worker.

7 Verlauf der einzelnen Läufe

7.1 Lauf 1 (`fibonacci-accumulator-cpu-c71b1f`)

Planung im ersten Versuch (2:39 min) mit 91 Einzelanforderungen. Implementierung, statischer Check, Simulation, Mutationsanalyse (Quote 0,9) und Abschlussprüfung ohne Befund, Synthese mit WNS +3,75 ns, Flash in SRAM und QSPI. Keine Reparatur. Beobachtet wurden die Verbesserungspunkte #48–#55 (Planung der Worker, 1,06 MB große `task_graph.json`, 101 KB großes Abschlussereignis, irreführende Auftragsgröße im Log, fehlende Testbench für cpu_core, Reset-Rückfrage aus einem Skill, Lücken in der Trajektorie, fehlende Prüfanleitung).

7.2 Lauf 2 (`mini-accumulator-cpu-66e33d`)

Wiederholung mit gleichem Codestand: gleiche Dauer (38,0 min), wieder ohne Reparatur. Unterschiede zeigen die Streuung des Sprachmodells: 74 statt 91 Einzelanforderungen, Mutationsquote 0,7 statt 0,9, cpu_core diesmal als „sequential“ eingestuft – und trotzdem ohne eigene Testbench, womit die tatsächliche Ursache gefunden war (Regel im HDLAgent, #52). Den auf das Board geflashten Entwurf dieses Laufs hat der Benutzer geprüft: die LED-Ausgabe stimmt.

7.3 Abgebrochener Kontrolllauf

> Ein erster Kontrolllauf mit den Verbesserungen #48–#55 (`fibonacci-accu-cpu-588373`) endete als BLOCKED: Gegen 16:43 lehnte der Anbieter hinter OpenCode Zen beide freien Modelle ab (`Upstream request failed: Invalid credential`, Gegenprobe außerhalb des Systems identisch, Dauer rund 1,5 h). Das System stufte den Ausfall als Entwurfsfehler ein und lief 13 Minuten lang die Eskalationsleiter hoch; die Planung startete nach dem ersten Ausfall noch vier weitere Module. Außerdem zeigte sich ein Folgefehler von #48 (Checkpoint als „Änderung durch OpenCode“). Behoben als #56; der Lauf ist nicht in den Kennzahlen enthalten.

7.4 Kontrolllauf (`fibonacci-accu-cpu-f8fb57`)

Planung mit Prüfanleitung (6 Schritte) und ohne offene Fragen; control_unit startete zuerst, cpu_core nach 6:18 min und bekam eine eigene Testbench. Der statische Check meldete dann eine Abweichung vom Referenzmodell: Bei Takt 27 erwartete das Modell in der PC-Anzeige 0011, der Entwurf zeigte 0010 – der ganze Ablauf lag einen Takt zurück. Ursache war `tick_gen`, das den Tick als Register erzeugte, also einen Takt nach dem Zählerstand TICK_CYCLES−1:

```
  p_seq : process(clk)
  begin
    if rising_edge(clk) then
      if rst = '1' then
        cnt  <= 0;
        tick <= '0';
      else
        if cnt = TICK_CYCLES - 1 then
          cnt  <= 0;
          tick <= '1';
        else
          cnt  <= cnt + 1;
          tick <= '0';
        end if;
      end if;
    end if;
  end process p_seq;
```

Die erste Reparatur fand die Stelle, bewertete sie aber falsch („reference_model … assumes combinational tick … DUT correctly shows pc=2“). Nach der Regel aus #40 wurde daraufhin das Referenzmodell mit diesem Befund neu erzeugt. Das neue Modell prüfte die Behauptung, wies sie unter Verweis auf den Wortlaut der Anforderung zurück und blieb bei der aufgabengetreuen Zeitfolge. Die zweite Reparatur korrigierte daraufhin den Entwurf:

```
src/tick_gen.vhd registered tick (tick='1' after 4th edge), so WAIT->FETCH occurred at 5th
edge and mini_cpu_model_tb lagged at cycle 27 (expected PC=3/led=0011 got PC=2/led=0010);
PROMPT R67 requires WAIT->FETCH at TICK_CYCLES-th edge, so tick changed to combinational
decode (cnt==TICK_CYCLES-1) and tests/tick_gen_tb.vhd expectations shifted from c mod T==0 to
c mod T==T-1.
```

```
tick <= '1' when cnt = TICK_CYCLES - 1 else '0';
```

Damit hat der Mechanismus „Modellstreit“ einen Entwurfsfehler nicht zum Modellfehler umgedeutet; die Runde kostete rund 13 Minuten. Am Board wäre der Fehler unsichtbar gewesen (10 ns), und auch die Referenz-Testbench mit ihrem Spielraum hätte ihn nicht erkannt. Danach Mutationsquote 1,0, 91 von 91 Anforderungen erfüllt, WNS +3,77 ns, Flash. Die Abschlussmeldung enthielt die Prüfanleitung – mit vertauschten Schaltern für ACC und PC in zwei Schritten, was zu #57 führte.

8 Befunde und Verbesserungen

Die Nummern entsprechen Abschnitt 15.1 in `docs/doku.html`, wo Anlass, Maßnahme, Dateien und Tests ausführlich stehen. Die Wirkung ist im Kontrolllauf gemessen bzw. mit den Dauern aus Lauf 1 nachgerechnet.

#	Maßnahme	gemessene Wirkung
48	Planung der parallelen Arbeit: kritischer Pfad zuerst, Start sobald Abhängigkeiten fertig	Module und Testbenches 20,5 statt 24,5 min; cpu_core startet nach 6:18 statt 13:09 min
49	task_graph.json ohne Kopien der Aufgabeneingaben	28 KB statt 1,06 MB
50	Abschlussereignis ohne Projektzustand, Lebenszeichen nach 60 s Stille	56 statt 163 KB Ereignisverkehr, 20 statt 103 Lebenszeichen, Abschlussereignis 1 statt 101 KB
51	OpenCode-Protokoll mit echter Auftragsgröße und Dauer	z. B. prompt_length 5944 statt immer 7516
52	Testbenches auch für Zwischenmodule mit eigener Logik	cpu_core_tb; Mutationsquote 1,0
53	Reset-Konvention der Skills folgt dem Manifest	keine offene Reset-Frage mehr im Plan
54	Trajektorie mit Planung, Modell, Modellwechsel und Endstand der Quellen	im Kontrolllauf vollständig erfasst
55	Prüfanleitung fürs Board aus dem Plan in der Abschlussmeldung	6 Schritte; siehe #57
56	Anbieterausfall als Infrastrukturfehler, Planung stoppt danach; Checkpoint zählt nicht als OpenCode-Änderung	ohne #56: 13 min Eskalation ohne Aussicht im ersten Kontrolllauf
57	Prüfanleitung wird von der Abschlussprüfung gegengeprüft	im Kontrolllauf waren in 2 Schritten SW0/SW1 vertauscht (vor #57)

Planung nach dem kritischen Pfad (#48). Mit den gemessenen Dauern aus Lauf 1 ergibt die neue Planung 21,2 statt 24,5 Minuten für die Module. Eine rein abhängigkeitsgesteuerte Planung ohne Priorität wäre mit 25,4 Minuten sogar langsamer gewesen als die alte paketweise; entscheidend ist die Regel „längste Kette zuerst“ (Listenplanung nach Hu 1961, Übersicht Kwok/Ahmad 1999).

Testbench für cpu_core (#52). Zwischenmodule mit eigener Logik bekommen jetzt eine Testbench; das kostet einen OpenCode-Aufruf (hier 2:19 min) und hob die Mutationsquote im Kontrolllauf auf 1,0.

Modellstreit. Die Regel aus #40 (Modell einmal neu erzeugen, erst beim zweiten Streit abschalten) hat sich bewährt: Das neu erzeugte Modell ist ein Schiedsrichter, der die Aufgabe liest, nicht den Entwurf.

9 Vergleich mit einem einzelnen OpenCode-Aufruf

Am 13.10.2026 wurde dieselbe Aufgabe mit demselben Modell als **ein einziger OpenCode-Aufruf** gelöst (Benchmark-Variante `single`, Lauf `20261013-072705`, Zeitlimit 3600 s). Der Auftrag lautet sinngemäß: PROMPT.md vollständig umsetzen, eine Testbench für das Top-Level schreiben, selbst mit GHDL prüfen und alle Fehler beheben. Synthese und Flash gehören nicht zur Variante; sie wurden danach auf einer Kopie mit dem FPGAAgent des MAS ausgeführt (gleiche Werkzeugkette), damit der Entwurf am Board geprüft werden kann.

	Single	**MAS (Lauf 1 / 2 / Kontrolllauf)**
Dauer bis simulierter Entwurf	**4,6 min**, 1 Aufruf	ca. 30 / 31 / 40 min, 22–24 Aufrufe
Dauer bis Board	**ca. 10,5 min** (mit Synthese und Flash)	37,7 / 38,0 / 47,1 min
Module (10 vorgegeben)	10	10 / 10 / 10
Testbenches	1 (Top-Level)	9 / 9 / 10
Unabhängige Referenz-Testbench	bestanden	bestanden (alle drei)
Taktgenauer Vergleich mit den drei MAS-Referenzmodellen (je 4 Zufallsfolgen)	12 von 12 übereinstimmend	Kontrolllauf: Abweichung von einem Takt gefunden und behoben
Mutationsquote (Mutationsanalyse des MAS, 10 Mutanten)	0,7	0,9 / 0,7 / 1,0
Timing (WNS)	+4,69 ns	+3,76 / +3,14 / +3,77 ns
LUTs / Flipflops	106 / 149	95/151, 108/149, 93/151
Einzelanforderungen mit Nachweis	keine	91 / 74 / 91, unabhängig bewertet

Korrektheit: Gleichstand. Der Einzelaufruf schrieb alle zehn Module in rund drei Minuten, die Testbench in vier, und der Entwurf ist taktgenau richtig – auch an der Stelle, an der das MAS im Kontrolllauf zunächst einen Takt zu spät lag. **Aufwand:** Single war etwa achtmal schneller und brauchte einen statt 22–24 Aufrufe. **Nachweise:** Hier liegt der Vorteil des MAS. Die einzige Testbench von Single ließ drei von zehn Mutanten durch, darunter einen HLT-Befehl ohne Wirkung; das MAS liefert je Modul eine Testbench, ein unabhängiges Referenzmodell und eine begründete Bewertung jeder Einzelanforderung.

> Einordnung: Bei dieser sehr genau spezifizierten, eng gekoppelten Aufgabe zeigt sich keine Überlegenheit des MAS bei der Lösungsquote; das deckt sich mit Befunden, nach denen Multi-Agenten-Systeme bei sequenziellen, gekoppelten Aufgaben eher schaden (Kim, Liu u. a. 2025). Der Mehrwert des MAS liegt in Nachweisbarkeit und Robustheit (Prüfungen, Reparatur, Weg bis zur Hardware ohne Zusatzschritte). Ob es darüber hinaus mehr leistet, müssen schwächer spezifizierte oder größere Aufgaben und mehrere Wiederholungen zeigen; ein Single-Lauf ist eine Stichprobe.

10 Grenzen

- Ob die LEDs am Board stimmen, prüft das System nicht selbst (kein UART); die Prüfanleitung muss ein Mensch abarbeiten und ist ein gegengeprüftes Sprachmodell-Urteil (#55, #57).
- Drei Läufe mit einem freien Modell sind keine statistische Aussage; Anforderungszahl, Testgüte und Dauer streuen zwischen Läufen.
- Die Referenz-Testbench prüft das feste Demoprogramm über die Ports; Fehler in selten genutzten Befehlsvarianten und Abweichungen um einzelne Takte erkennt sie nicht.
- Anbieterausfälle beendet das System mit einer Meldung (#56); es wartet nicht selbst, bis der Anbieter wieder erreichbar ist.
- Eine echte 8-Bit-CPU (8-Bit-Datenpfad, größerer Speicher, UART-Testprogramm) steht noch aus.

11 Ablage und Reproduktion

Pfad	Inhalt
`Schaltungsagent/benchmark/prompts/04_mini_cpu.md`	Aufgabe
`Schaltungsagent/benchmark/reference/04_mini_cpu_ref_tb.vhd`	unabhängige Referenz-Testbench
`docs/benchmark/ergebnisse/20261012-04_mini_cpu/`	results.csv, summary.json, meta.json, HINWEIS.md
`… /laufN_<projekt>/`	Plan, Quellen, Testbenches, Modell, Berichte, Ereignisse, Log, Trajektorie, Kennzahlen
`… /pruefgrundlagen/`	Befehlssatzmodell (Python) und eigene Vergleichsumsetzung (VHDL)
`docs/benchmark/ergebnisse/20261013-072705/`	Vergleichslauf Single: Kennzahlen, Entwurf, Synthese
`docs/benchmark/tools/aufgabe04_dokumentation.py`	erzeugt dieses Dokument

```
# Referenz-Testbench gegen einen gesicherten Entwurf (Beispiel Lauf 1)
cd "$(mktemp -d)"
ghdl -i --std=08 <repo>/docs/benchmark/ergebnisse/20261012-04_mini_cpu/lauf1_*/src/*.vhd \
     <repo>/Schaltungsagent/benchmark/reference/04_mini_cpu_ref_tb.vhd
ghdl -m --std=08 ref_mini_cpu_tb && ghdl -r --std=08 ref_mini_cpu_tb --stop-time=1ms --assert-
level=error
```

Literatur

- T. C. Hu: Parallel Sequencing and Assembly Line Problems. Operations Research 9(6), 1961, S. 841–848.
- Y.-K. Kwok, I. Ahmad: Static Scheduling Algorithms for Allocating Directed Task Graphs to Multiprocessors. ACM Computing Surveys 31(4), 1999, S. 406–471.
- E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, S. Yoo: The Oracle Problem in Software Testing: A Survey. IEEE Transactions on Software Engineering 41(5), 2015, S. 507–525.
- Y. Kim, X. Liu u. a. (Google Research): Towards a Science of Scaling Agent Systems. arXiv:2512.08296, 2025.

- J. Pan u. a.: Training Software Engineering Agents and Verifiers with SWE-Gym. arXiv:2412.21139, 2024.